

Rapport d'Évaluation du Modèle

Projet : API d'Analyse de Sentiments (SocialMetrics AI) [cite: 1]

Client : Daunale Treupe [cite: 1]

1. Contexte et Méthodologie

Ce rapport présente les résultats de l'évaluation du modèle de Machine Learning basé sur la régression logistique, déployé via une API Flask pour le compte de SocialMetrics AI [cite: 1]. L'objectif est de surveiller les opinions exprimées sur le réseau social X en évaluant le sentiment des tweets [cite: 1].

Logique de l'API : Conformément au code déployé, l'API traite un tableau de chaînes de caractères (string[]) [cite: 1]. Pour chaque tweet, le score final est calculé selon la formule :

$$\text{Score} = P(\text{positive}) - P(\text{negative})$$

Ce score renvoie une valeur continue qui reflète l'intensité du sentiment.

2. Performances du Modèle : Classe "Positive"

Le modèle a été évalué sur un jeu de données de validation afin de prédire si un tweet est jugé positif (1) ou non (0) [cite: 1].

Rapport de Classification (Positif)

Classe	Précision	Rappel	F1-Score	Support
0 (Non Positif)	1.00	1.00	1.00	19
1 (Positif)	1.00	1.00	1.00	21
Global (Accuracy)	1.00			40

Matrice de Confusion (Positif) [cite: 1]

Vrais Négatifs (TN) 19	Faux Positifs (FP) 0
Faux Négatifs (FN) 0	Vrais Positifs (TP) 21

3. Performances du Modèle : Classe "Négative"

Ce second modèle indépendant évalue si un tweet est jugé négatif (1) ou non (0) [cite: 1].

Rapport de Classification (Négatif)

Classe	Précision	Rappel	F1-Score	Support
0 (Non Négatif)	1.00	1.00	1.00	21
1 (Négatif)	1.00	1.00	1.00	19
Global (Accuracy)	1.00			40

Matrice de Confusion (Négatif) [cite: 1]

Vrais Négatifs (TN) 21	Faux Positifs (FP) 0
Faux Négatifs (FN) 0	Vrais Positifs (TP) 19

4. Analyse des Performances et Biais Éventuels [cite: 1]

Les métriques de **précision**, **rappel** et **F1-score** atteignent toutes un score parfait de **1.00 (100%)** pour les deux classes [cite: 1]. Bien que cela puisse sembler idéal en théorie, dans un contexte de traitement du langage naturel (NLP) appliqué aux réseaux sociaux, un tel résultat est particulièrement atypique.

Biais identifiés :

- **Surapprentissage (Overfitting) / Fuite de données (Data Leakage)** : Les modèles ont potentiellement mémorisé les données. Il est fort probable que les données de validation fassent également partie de l'ensemble d'entraînement, ou que le dataset soit trop simpliste.
- **Manque de complexité sémantique** : Les tweets réels contiennent généralement de l'ironie, du sarcasme, des fautes d'orthographe et du jargon. Un score de 100% indique que le jeu de test actuel ne capture pas cette complexité réelle.

- **Taille de l'échantillon** : Le support évalué (40 échantillons par modèle) est statistiquement trop faible pour valider la robustesse générale de l'algorithme sur des données de production massives.

5. Recommandations et Pistes d'Amélioration [cite: 1]

Afin de garantir que l'API réponde efficacement aux attentes en conditions réelles, les actions suivantes sont recommandées [cite: 1] :

- **Séparation stricte des jeux de données** : S'assurer que le dataset de validation est totalement indépendant du dataset d'entraînement.
- **Enrichissement des données (Data Augmentation)** : Collecter et annoter un corpus de tweets beaucoup plus vaste et diversifié, incluant des cas limites (tweets neutres, ironiques, multilingues).
- **Optimisation du prétraitement (Preprocessing)** : Mettre en œuvre des techniques avancées via le vectorizer (suppression des stopwords, lemmatisation, gestion des emojis qui sont cruciaux pour l'analyse de sentiment).
- **Validation croisée (Cross-validation)** : Utiliser des techniques comme la K-Fold Cross Validation pour obtenir une estimation plus réaliste et stable des performances de la régression logistique [cite: 1].
- **Surveillance post-déploiement** : Puisque le modèle est réentraîné régulièrement via cronjob avec les nouvelles données de la table tweets [cite: 1], il est impératif de logger les faux positifs/négatifs rencontrés par les utilisateurs en production pour ajuster le modèle.